

Flappy Bird — Lab 4 Chat History

Formatted from the conversation text provided

Fix Flappy Bird pipe collisions

Here's my Flappy Bird code. The bird sometimes passes through pipe edges without dying, especially at higher speeds. Find the cause in the collision logic and fix it. Use rect-based collision (`pygame.Rect.colliderect`) for both the top and bottom pipes, and explain what was wrong. "Space/click before gravity starts."

User context

Attachments:

- `main.py: `./40_flappy-bird/main.py``
- `README.md: `./40_flappy-bird/README.md``
- `bird.py: `./40_flappy-bird/game/bird.py``
- `game_engine.py: `./40_flappy-bird/game/game_engine.py``
- `pipe.py: `./40_flappy-bird/game/pipe.py``

<details><summary>12 previous messages</summary>

I'll inspect the Flappy Bird files first, especially the pipe and engine collision path, then I'll patch the logic in place and run whatever lightweight checks the project supports.

<details><summary>Explored 5 files</summary>

- `Read `./40_flappy-bird/game/game_engine.py``
- `Read `./40_flappy-bird/game/bird.py``
- `Read `./40_flappy-bird/game/pipe.py``
- `Read `./40_flappy-bird/main.py``
- `Read `./40_flappy-bird/README.md``

</details>

The bug is right where the comment says: the current code asks "is the bird's center inside a pipe?" so grazing the edge with the bird's radius never counts. I'm going to switch that to comparing the bird's full bounding rect against each pipe rect.

<details><summary>Edited a file</summary>

- `Wrote `./40_flappy-bird/game/game_engine.py` (+2 -6)`

</details>

Patch is in. I'm doing a quick sanity check now with a small headless collision scenario so we verify both top and bottom pipe collisions trigger when only the bird's outer rect, not its center point, overlaps.

<details><summary>Explored a file, a list, ran 4 commands</summary>

<details><summary>Explored a file, a list</summary>

- `Read `./40_flappy-bird/requirements.txt``
- `Listed files in `40_flappy-bird``

</details>

- `Ran `git diff -- 40_flappy-bird/game/game_engine.py``
- `Ran `pwd``
- `Ran `git diff -- game/game_engine.py``

```
• Ran `python3 -c $'import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom game.game_engine\nimport GameEngine\nfrom game.pipe import Pipe\nengine = GameEngine(500, 700)\nengine.pipe_speed = 20\nengine.pipes =\n[Pipe(110, 700, gap=150, width=70, speed=0)]\nengine.pipes[0].gap_y = 200\nengine.bird.x = 95\nengine.bird.y =\n100\nengine.bird.velocity = 0\nengine.update()\nassert engine.game_over, \"top pipe edge collision was not detected\"\nengine =\nGameEngine(500, 700)\nengine.pipe_speed = 20\nengine.pipes = [Pipe(110, 700, gap=150, width=70, speed=0)]\nengine.pipes[0].gap_y\n= 200\nengine.bird.x = 95\nengine.bird.y = 365\nengine.bird.velocity = 0\nengine.update()\nassert engine.game_over, \"bottom pipe edge\ncollision was not detected\"\nprint(\"rect collision sanity checks passed\")\npygame.quit()`
```

</details>

The code change is clean, but the local Python I invoked doesn't have `pygame` installed, so the sanity check couldn't import the game. I'll look for an existing project virtualenv before deciding whether dependency install is needed.

<details><summary>Explored a search, a list, ran a command</summary>

<details><summary>Explored a search, a list</summary>

- Searched for `.venv` in `LAB4`

- Listed files

</details>

- Ran `python3 --version`

</details>

There is a project `venv`, so I'll use that rather than touching dependencies. One more check, this time through the environment the project already has.

<details><summary>Explored a file, ran a command</summary>

<details><summary>Explored a file</summary>

- Read `./game/game\_engine.py`

</details>

```
• Ran `./venv/bin/python -c $'import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom\n game.game_engine import GameEngine\nfrom game.pipe import Pipe\nengine = GameEngine(500, 700)\nengine.pipes = [Pipe(110, 700,\ngap=150, width=70, speed=0)]\nengine.pipes[0].gap_y = 200\nengine.bird.x = 95\nengine.bird.y = 100\nengine.bird.velocity =\n0\nengine.update()\nassert engine.game_over, \"top pipe edge collision was not detected\"\nengine = GameEngine(500,\n700)\nengine.pipes = [Pipe(110, 700, gap=150, width=70, speed=0)]\nengine.pipes[0].gap_y = 200\nengine.bird.x = 95\nengine.bird.y =\n365\nengine.bird.velocity = 0\nengine.update()\nassert engine.game_over, \"bottom pipe edge collision was not detected\"\nprint(\"rect\ncollision sanity checks passed\")\npygame.quit()`
```

</details>

One adjacent behavior in your request is that gravity should wait for the first Space/click. I'm making that tiny state change too: input starts the round and flaps; until then, update holds the world still.

<details><summary>Edited a file, ran 2 commands</summary>

- Wrote `./40\_flappy-bird/game/game\_engine.py` (+10 -3)

- Ran `git diff -- game/game\_engine.py`

```
• Ran `./venv/bin/python -c $'import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom\n game.game_engine import GameEngine\nfrom game.pipe import Pipe\nengine = GameEngine(500, 700)\nstarting_y =\nengine.bird.y\nengine.update()\nassert engine.bird.y == starting_y, \"bird moved before the first input\"\nengine.started =\nTrue\nengine.pipes = [Pipe(110, 700, gap=150, width=70, speed=0)]\nengine.pipes[0].gap_y = 200\nengine.bird.x = 96\nengine.bird.y =\n100\nengine.bird.velocity = 0\nengine.update()\nassert engine.game_over, \"top pipe edge collision was not detected\"\nengine =\nGameEngine(500, 700)\nengine.started = True\nengine.pipes = [Pipe(110, 700, gap=150, width=70, speed=0)]\nengine.pipes[0].gap_y =\n200\nengine.bird.x = 96\nengine.bird.y = 336\nengine.bird.velocity = 0\nengine.update()\nassert engine.game_over, \"bottom pipe edge\ncollision was not detected\"\nprint(\"rect collision and start gate sanity checks passed\")\npygame.quit()`
```

</details>

</details>

Fixed in [game_engine.py](./40_flappy-bird/game/game_engine.py:21).

What was wrong: the old collision used `collidepoint(self.bird.center())`, so only the bird's center had to enter a pipe. If the bird's circular/rect body clipped a pipe edge but its center stayed outside, no death was triggered. Higher speeds made that easier to notice because objects move farther per frame.

What changed:

- Added `bird_rect = self.bird.rect()`
- Checked both pipes with `bird_rect.collidirect(pipe.top_rect())` and `bird_rect.collidirect(pipe.bottom_rect())`
- Added the requested start gate: Space/click starts the round, and gravity/world updates wait until then.

Verified with the project `venv`: headless sanity checks passed for top pipe edge collision, bottom pipe edge collision, and "no gravity before first input."

Task 2: Game Over screen

- Example prompt: **"Add a game over state. When the bird hits the ground, ceiling or a pipe, show a screen with 'Game Over' and the final score drawn in the Pygame window, then wait for input instead of printing to the console."**

<details><summary>8 previous messages</summary>

I'll build this directly into the existing engine: keep the game loop alive after death, draw an overlay with the final score, and remove the console-only game-over behavior. I'll also preserve the start-on-input behavior from Task 1.

<details><summary>Explored 2 files, ran a command</summary>

<details><summary>Explored 2 files</summary>

- Read `./main.py`
- Read `./game/game_engine.py`

</details>

- Ran `git status --short`

</details>

I found the current `render()` still prints only once on game over. I'm replacing that with a drawn overlay and a small event return so the main loop can stay open on the game-over screen until the player presses a key/clicks to exit.

<details><summary>Edited 2 files</summary>

- Wrote `./40_flappy-bird/game/game_engine.py` (+27 -4)
- Wrote `./40_flappy-bird/main.py` (+2 -1)

</details>

The overlay and event flow are patched. I'm running a quick headless render/event check now: it should draw the game-over screen without printing, and a key press after death should tell the loop to quit.

<details><summary>Explored 2 files, ran 2 commands</summary>

- Ran `git diff -- game/game_engine.py main.py`
- Ran `./venv/bin/python -c $'import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom game.game_engine import GameEngine\nengine = GameEngine(500, 700)\nscreen = pygame.Surface((500, 700))\nengine.game_over = True\nengine.score = 7\nengine.render(screen)\nresult = engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_SPACE))\nassert result == \"quit\", result\nprint(\"game over screen sanity check passed\")\npygame.quit()'`

<details><summary>Explored 2 files</summary>

- Read `./main.py`
- Read `./game/game_engine.py`

</details>

</details>

One tiny cleanup before I wrap: I noticed an unused color constant from the overlay edit, so I'm removing that stray line to keep the file tidy.

<details><summary>Edited a file, ran 2 commands</summary>

- Wrote `./40_flappy-bird/game/game_engine.py` (+0 -1)`
- Ran `./venv/bin/python -c '$import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom game.game_engine import GameEngine\nengine = GameEngine(500, 700)\nscreen = pygame.Surface((500, 700))\nengine.game_over = True\nengine.score = 7\nengine.render(screen)\nresult = engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_SPACE))\nassert result == \"quit\", result\nprint(\"game over screen sanity check passed\")\npygame.quit()'`
- Ran `git diff -- game/game_engine.py main.py``

</details>

</details>

Implemented Task 2.

The game now shows an in-window Game Over overlay with the final score instead of printing to the console. After the bird hits a pipe, ceiling, or ground, `update()` freezes the game state, `render()` draws the overlay, and the window waits until the player presses any key or clicks to exit.

Changed:

- `[game_engine.py](./40_flappy-bird/game/game_engine.py:20)`: added game-over fonts and `render_game_over()`
- `[game_engine.py](./40_flappy-bird/game/game_engine.py:29)`: game-over input now returns `\"quit\"`
- `[main.py](./40_flappy-bird/main.py:30)`: main loop exits cleanly when game-over input is received

Verified with a headless Pygame sanity check: overlay renders and post-game input is handled.

Task 3: Replay with difficulty

- Example prompt: `\"On the game over screen, let the player press a key to choose Easy, Medium or Hard (different pipe speed and gap size) to replay, or Q/Esc to quit. Reset the game state properly on replay.\"`

<details><summary>6 previous messages</summary>

I'll extend the existing game-over state into a little difficulty picker: replay keys will rebuild the bird, pipes, score, timers, and pipe settings; quit keys will still close the window cleanly.

<details><summary>Explored 3 files</summary>

- Read `./game/pipe.py``
- Read `./game/game_engine.py``
- Read `./main.py``

</details>

The cleanest shape here is to give `GameEngine`` a `reset()` method and make difficulty a small config table. I'm patching that now so replay uses the same initialization path every time, which avoids half-reset state hanging around.

<details><summary>Edited a file</summary>

- Wrote `./40_flappy-bird/game/game_engine.py` (+34 -14)`

</details>

Replay is wired in. I'm going to verify the reset path now: score/timers/game-over should clear, the selected difficulty should set speed and gap, and Q/Esc should still return a quit signal.

<details><summary>Explored a file, ran 2 commands</summary>

<details><summary>Explored a file</summary>

- Read `./game/game_engine.py``

</details>

- Ran ``git diff -- game/game_engine.py main.py``

- Ran ``./env/bin/python -c $'import os\nos.environ[\"SDL_VIDEODRIVER\"] = \"dummy\"\nimport pygame\npygame.init()\nfrom game.game_engine import GameEngine\nengine = GameEngine(500, 700)\nassert engine.pipe_speed == 4\nassert engine.pipe_gap == 150\nassert engine.difficulty_name == \"Medium\"\nengine.score = 12\nengine._spawn_timer = 44\nengine.started = True\nengine.game_over = True\nengine.pipes[0].scored = True\nengine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_h))\nassert not engine.game_over\nassert not engine.started\nassert engine.score == 0\nassert engine._spawn_timer == 0\nassert engine.pipe_speed == 6\nassert engine.pipe_gap == 120\nassert engine.difficulty_name == \"Hard\"\nassert len(engine.pipes) == 1\nassert engine.pipes[0].speed == 6\nassert engine.pipes[0].gap == 120\nengine.game_over = True\nassert engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_ESCAPE)) == \"quit\"\nengine.game_over = True\nassert engine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_q)) == \"quit\"\nscreen = pygame.Surface((500, 700))\nengine.render(screen)\nprint(\"replay difficulty sanity checks passed\")\npygame.quit()``

</details>

</details>

Implemented Task 3 in [game_engine.py](./40_flappy-bird/game/game_engine.py:10).

The game-over screen now lets the player choose:

- ``E`` for Easy: speed ``3``, gap ``180``
- ``M`` for Medium: speed ``4``, gap ``150``
- ``H`` for Hard: speed ``6``, gap ``120``
- ``Q`` or ``Esc`` to quit

Replay resets the bird, pipes, score, spawn timer, ``game_over``, and ``started`` state cleanly through a new ``reset()`` method. New pipes now use the selected difficulty's speed and gap.

Verified with a headless Pygame sanity check for difficulty replay, full state reset, render, and quit keys.

****Task 4: Sound effects****

- Example prompt: `**Add sound effects for flapping, scoring and dying using pygame.mixer. Load them safely so the game still runs if a sound file is missing.**`
- Put the `.wav`` files in a ``sounds/`` folder (free ones from freesound.org or kenney.nl), and commit them with the code.

<details><summary>14 previous messages</summary>

I'll add the mixer path defensively: initialize/load sounds if possible, otherwise fall back to silent no-ops. For the `.wav`` files, I'll generate tiny original WAV cues locally in ``sounds/`` so the project has committed assets without depending on a download.

<details><summary>Explored a file, a list, ran a command</summary>

<details><summary>Explored a file, a list</summary>

- Read ``./game/game_engine.py``
- Listed files

</details>

- Ran ``git status --short``

</details>

The sound hooks need to be placed at state changes, not every frame: flap input, first score pass, and the exact moment death flips ``game_over``. I'm adding a tiny safe loader plus ``play_sound()/end_game()`` helpers so missing audio or mixer failures won't crash gameplay.

<details><summary>Edited a file</summary>

- Wrote ``./40_flappy-bird/game/game_engine.py`` (+45 -2)

</details>

Code is patched. Now I'm creating three small WAV files in ``sounds/``: a quick upward chirp for flap, a bright two-tone score cue, and a short descending death cue.

<details><summary>Ran 2 commands</summary>

- Ran ``mkdir -p sounds``
- Ran ``python3 -c $'import math\nimport wave\nimport struct\nfrom pathlib import Path\n\nSAMPLE_RATE = 44100\nAMPLITUDE = 14000\n\ndef write_tone(path, segments):\n frames = []\n phase = 0.0\n for start_freq, end_freq, duration, volume in segments:\n count = int(SAMPLE_RATE * duration)\n for i in range(count):\n t = i / max(count - 1, 1)\n freq = start_freq + (end_freq - start_freq) * t\n phase += 2 * math.pi * freq / SAMPLE_RATE\n envelope = min(1.0, i / max(int(SAMPLE_RATE * 0.01), 1), (count - i) / max(int(SAMPLE_RATE * 0.03), 1))\n sample = int(math.sin(phase) * AMPLITUDE * volume * envelope)\n frames.append(struct.pack("<h", sample))\n with wave.open(str(path), "wb") as wav:\n wav.setnchannels(1)\n wav.setsampwidth(2)\n wav.setframerate(SAMPLE_RATE)\n wav.writeframes(b"".join(frames))\n\nsounds = Path("sounds")\nwrite_tone(sounds / "flap.wav", [(520, 920, 0.12, 0.55)])\nwrite_tone(sounds / "score.wav", [(740, 740, 0.08, 0.45), (980, 1180, 0.12, 0.45)])\nwrite_tone(sounds / "die.wav", [(360, 120, 0.32, 0.60)])\nprint("created", ", ".join(p.name for p in sorted(sounds.glob("*wav")))``

</details>

The assets are in place. I'm running checks with Pygame's dummy video/audio drivers so we can confirm sounds load when available and the game still behaves without opening a window.

<details><summary>Ran 4 commands</summary>

- Ran ``file sounds/*.wav``
- Ran ``SDL_VIDEODRIVER=dummy SDL_AUDIODRIVER=dummy ./venv/bin/python -c $'import pygame\npygame.init()\nfrom game.game_engine import GameEngine\nfrom game.pipe import Pipe\nengine = GameEngine(500, 700)\nassert all(engine.sounds[name] is not None for name in ("flap", "score", "die"))\nengine.handle_event(pygame.event.Event(pygame.KEYDOWN, key=pygame.K_SPACE))\nassert engine.started\nengine.pipes = [Pipe(0, 700, gap=150, speed=0)]\nengine.pipes[0].x = -80\nengine.pipes[0].scored = False\nengine.update()\nassert engine.score == 1\nengine.end_game()\nassert engine.game_over\nprint("sound sanity checks passed")\npygame.quit()``
- Ran ``./venv/bin/python -m py_compile main.py game/game_engine.py game/bird.py game/pipe.py``
- Ran ``git status --short``

</details>

The checks passed, but ``py_compile`` touched tracked ``__pycache__`` files in this repo. I'm going to restore those generated cache files from git so the actual deliverable stays focused on the source and sound assets.

<details><summary>Explored a list, ran 3 commands</summary>

- Ran ``git restore game/__pycache__/bird.cpython-313.pyc game/__pycache__/game_engine.cpython-313.pyc game/__pycache__/pipe.cpython-313.pyc``

<details><summary>Explored a list</summary>

- Listed files in ``__pycache__``

</details>

- Ran ``rm __pycache__/main.cpython-313.pyc``
- Ran ``rmdir __pycache__``

</details>

Generated cache files are cleaned up. I'm doing one final status/diff pass so I can report exactly what changed.

<details><summary>Explored a list, ran 2 commands</summary>

- Ran ``git diff -- game/game_engine.py``
- Ran ``git status --short``

<details><summary>Explored a list</summary>

- Listed files in ``sounds``

</details>

</details>

Since you explicitly asked to commit the sound files with the code, I'll create a focused git commit containing `game\_engine.py` and the new `sounds/` WAV assets.

<details><summary>Ran 5 commands</summary>

- Ran `git add game/game\_engine.py sounds/flap.wav sounds/score.wav sounds/die.wav`
- Ran `git diff --check`
- Ran `git commit -m "Add sound effects"`
- Ran `git status --short`
- Ran `git show --stat --oneline --name-only HEAD`

</details>

</details>

Implemented Task 4 and committed it.

Added safe sound support in [game_engine.py](./40_flappy-bird/game/game_engine.py:1):

- Flap sound on Space/click
- Score sound when passing a pipe
- Death sound when game over starts
- Safe `pygame.mixer` loading so missing files or audio-device issues do not crash the game

Added WAV assets:

- [flap.wav](./40_flappy-bird/sounds/flap.wav)
- [score.wav](./40_flappy-bird/sounds/score.wav)
- [die.wav](./40_flappy-bird/sounds/die.wav)

Verified with dummy Pygame video/audio drivers and `py\_compile`.

Commit created: `aadfa40 Add sound effects`